



707. Design Linked List

Description:

- ✓ Design your implementation of the linked list.
- ✓ You can choose to use a singly or doubly linked list.
- ✓ A node in a singly linked list should have two attributes: val and next.
- ✓ val is the value of the current node, and next is a pointer/reference to the next node.
- ✓ If you want to use the doubly linked list, you will need one more attribute prev to indicate the previous node in the linked list.
- ✓ Assume all nodes in the linked list are 0-indexed.

Implement the MyLinkedList class:

- a) **MyLinkedList()** :
 - ✓ Initializes the MyLinkedList object.
- b) **int get(int index)** :
 - ✓ Get the value of the indexth node in the linked list. If the index is invalid, return -1.
- c) **void addAtHead(int val)** :
 - ✓ Add a node of value val before the first element of the linked list. After the insertion, the new node will be the first node of the linked list.
- d) **void addAtTail(int val)** :
 - ✓ Append a node of value val as the last element of the linked list.
- e) **void addAtIndex(int index, int val)** :
 - ✓ Add a node of value val before the indexth node in the linked list. If index equals the length of the linked list, the node will be appended to the end of the linked list. If index is greater than the length, the node will not be inserted.
- f) **void deleteAtIndex(int index)** :
 - ✓ Delete the indexth node in the linked list, if the index is valid.

Example 1:

```
MyLinkedList myLinkedList = new MyLinkedList();
myLinkedList.addAtHead(1);
myLinkedList.addAtTail(3);
myLinkedList.addAtIndex(1, 2); // linked list becomes 1->2->3
myLinkedList.get(1);           // return 2
myLinkedList.deleteAtIndex(1); // now the linked list is 1->3
myLinkedList.get(1);           // return 3
```

Constraints:

- a) $0 \leq \text{index}, \text{val} \leq 1000$
- b) Please do not use the built-in LinkedList library.
- c) At most 2000 calls will be made to get, addAtHead, addAtTail, addAtIndex and deleteAtIndex.

Companies

6 months - 1 year

1 year - 2 years

Google 3

Microsoft 3

Virtu Financial 3

Companies

6 months - 1 year

1 year - 2 years

Apple 3

Amazon 2



707. Design Linked List - Solution

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

class Node {
    int val;
    Node next;

    Node() {
    }

    Node(int val) {
        this.val = val;
    }
}

class MyLinkedList {

    int size;
    Node headNode;

    public MyLinkedList() {
        this.size = 0;
        this.headNode = new Node(0); // Sentinel Node
    }

    public void addAtHead(int val) {
        addAtIndex(0, val);
    }

    public void addAtTail(int val) {
        addAtIndex(size, val);
    }
}
```



```
public void addAtIndex(int index, int val) { // (0,10)

    if (index < 0 || index > size) {
        return;
    }

    Node temp = new Node(val);

    Node currNode = this.headNode;
    for(int i=0;i<index ;i++) {
        currNode = currNode.next;
    }

    temp.next = currNode.next;
    currNode.next = temp;
    size++;

}

public int get(int index) {
    if (index < 0 || index >= size) {
        return -1;
    }

    Node currNode = this.headNode;
    for(int i=0;i<index+1 ;i++) {
        currNode = currNode.next;
    }

    return currNode.val;
}
```



```
public void deleteAtIndex(int index) {  
  
    if (index < 0 || index >= size) {  
        return;  
    }  
  
    Node currNode = this.headNode;  
    for(int i=0;i<index ;i++) {  
        currNode = currNode.next;  
    }  
  
    Node temp = currNode.next;  
    currNode.next = currNode.next.next;  
    temp.next=null;  
  
    size--;  
  
}  
}
```



2. Add Two Numbers

Description:

- ✓ You are given two non-empty linked lists representing two non-negative integers.
- ✓ The digits are stored in reverse order, and each of their nodes contains a single digit.
- ✓ Add the two numbers and return the sum as a linked list.
- ✓ You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example 1:

Input: l1 = [2,4,3], l2 = [5,6,4]

Output: [7,0,8]

Explanation: $342 + 465 = 807$.

Example 2:

Input: l1 = [0], l2 = [0]

Output: [0]

Example 3:

Input: l1 = [9,9,9,9,9,9,9], l2 = [9,9,9,9]

Output: [8,9,9,9,0,0,0,1]

Constraints:

- The number of nodes in each linked list is in the range [1, 100].
- $0 \leq \text{Node.val} \leq 9$
- It is guaranteed that the list represents a number that does not have leading zeros.



Java Learning Center

No.1 In Java Training & placement

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Adobe

8

Amazon

8

Apple

6

Microsoft

4

Bloomberg

3

Expedia

3

Facebook

2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Google

8

Uber

7

Oracle

5

IBM

5

Cisco

3

Yahoo

2

SAP

2

PayTM

2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Capital One

4

JPMorgan

3

Yandex

3

Huawei

3

Cognizant

3

Twitter

2

Samsung

2

eBay

2

Goldman Sachs

2

Morgan Stanley

2

Paypal

2

Visa

2

ServiceNow

2

tcs

2

Airbnb

Wix



2. Add Two Numbers - Solution

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

class Solution {

    public ListNode addTwoNumbers(ListNode list1, ListNode list2) {
        ListNode headNode = new ListNode(0);
        ListNode curr = headNode;
        int carry = 0;
        while (list1 != null || list2 != null || carry != 0) {

            int x = (list1 != null) ? list1.val : 0;
            int y = (list2 != null) ? list2.val : 0;

            int sum = carry + x + y;
            carry = sum / 10;
            curr.next = new ListNode(sum % 10);
            curr = curr.next;

            if (list1 != null)
                list1 = list1.next;

            if (list2 != null)
                list2 = list2.next;
        }

        return headNode.next;
    }
}
```

876. Middle of the Linked List

Description:

- ✓ Given the head of a singly linked list, return the middle node of the linked list.
- ✓ If there are two middle nodes, return the second middle node.

Example 1:

Input: head = [1,2,3,4,5]

Output: [3,4,5]

Explanation: The middle node of the list is node 3.

Example 2:

Input: head = [1,2,3,4,5,6]

Output: [4,5,6]

Explanation: Since the list has two middle nodes with values 3 and 4, we return the second one.

Constraints:

- The number of nodes in the list is in the range [1, 100].
- $1 \leq \text{Node.val} \leq 100$

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Adobe 2

Oracle 2

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Amazon **5**

Apple **4**

Facebook **3**

Qualcomm **2**

Google **2**

Bloomberg **2**

Uber **2**

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Microsoft **6**

Samsung **2**

Goldman Sachs **2**

JPMorgan **2**

Nvidia **2**

Walmart Global Tech

876. Middle of the Linked List - Solution

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public ListNode middleNode(ListNode headNode) {

        ListNode slow = headNode;
        ListNode fast = headNode;

        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow;
    }
}
```

2095. Delete the Middle Node of a Linked List

Description:

- ✓ You are given the head of a linked list.
- ✓ Delete the middle node, and return the head of the modified linked list.

Example 1:

Input: head = [1,3,4,7,1,2,6]

Output: [1,3,4,1,2,6]

Example 2:

Input: head = [1,2,3,4]

Output: [1,2,4]

Example 3:

Input: head = [2,1]

Output: [2]

Constraints:

- The number of nodes in the list is in the range [1, 105].
- $1 \leq \text{Node.val} \leq 105$

Companies

6 months - 1 year

1 year - 2 years

Adobe 2



Companies

6 months - 1 year

1 year - 2 years

Microsoft **2**

2095. Delete the Middle Node of a Linked List

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public ListNode deleteMiddle(ListNode headNode) {

        if(headNode.next==null) {
            return null;
        }

        ListNode slow = headNode;
        ListNode fast = headNode.next.next;

        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }

        ListNode temp = slow.next;
        slow.next = slow.next.next;
        temp.next = null;

        return headNode;
    }
}
```



203. Remove Linked List Elements

Description:

- ✓ Given the head of a linked list and an integer val, remove all the nodes of the linked list that has `Node.val == val`, and return the new head.

Example 1:

Input: head = [1,2,6,3,4,5,6], val = 6
Output: [1,2,3,4,5]

Example 2:

Input: head = [], val = 1
Output: []

Example 3:

Input: head = [7,7,7,7], val = 7
Output: []

Constraints:

- a) The number of nodes in the list is in the range [0, 104].
- b) $1 \leq \text{Node.val} \leq 50$
- c) $0 \leq \text{val} \leq 50$

Companies

6 months - 1 year 1 year - 2 years

Google 4

Adobe 3

Apple 3

Amazon 2

Bloomberg 2



Companies

6 months - 1 year

1 year - 2 years

Facebook 7

Microsoft 4

203. Remove Linked List Elements - Solution

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public ListNode removeElements(ListNode headNode, int myval) {

        ListNode dummyHead = new ListNode(0);
        dummyHead.next = headNode;

        ListNode currNode = headNode;
        ListNode prevNode = dummyHead;

        while(currNode!=null) {
            if(currNode.val==myval) {
                prevNode.next = currNode.next;
            }else {
                prevNode=currNode;
            }
            currNode = currNode.next;
        }

        return dummyHead.next;
    }
}
```



237. Delete the Node in Linked List

Description:

- ✓ There is a singly-linked list **head** and we want to delete a node **node** in it.
- ✓ You are given the node to be deleted **node**. You will not be given access to the first node of **head**.
- ✓ All the values of the linked list are unique, and it is guaranteed that the given node **node** is not the last node in the linked list.
- ✓ Delete the given node. Note that by deleting the node, we do not mean removing it from memory. We mean:
 - The value of the given node should not exist in the linked list.
 - The number of nodes in the linked list should decrease by one.
 - All the values before node should be in the same order.
 - All the values after node should be in the same order.

Example 1:

Input: head = [4,5,1,9], node = 5
Output: [4,1,9]

Example 2:

Input: head = [4,5,1,9], node = 1
Output: [4,5,9]

Constraints:

- a. The number of the nodes in the given list is in the range [2, 1000].
- b. $-1000 \leq \text{Node.val} \leq 1000$
- c. The value of each node in the list is unique.
- d. The node to be deleted is in the list and is not a tail node.



Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Amazon **3**

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Bloomberg **3**

Apple **2**

Adobe **2**

Google **2**

Nagarro **2**

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Microsoft **6**

Oracle **2**

Yahoo **2**

Qualcomm **2**

Goldman Sachs **2**

Morgan Stanley **2**

237. Delete the Node in Linked List - Solution

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public void deleteNode(ListNode node) {

        ListNode nextNode = node.next;
        node.val=nextNode.val;
        node.next = nextNode.next;
        nextNode.next =null;

    }
}
```



19. Remove Nth Node From End of List

Description:

- ✓ Given the head of a linked list, remove the nth node from the end of the list and return its head.

Example 1:

Input: head = [1,2,3,4,5], n = 2
Output: [1,2,3,5]

Example 2:

Input: head = [1], n = 1
Output: []

Example 3:

Input: head = [1,2], n = 1
Output: [1]

Constraints:

- a) The number of nodes in the list is sz.
- b) $1 \leq sz \leq 30$
- c) $0 \leq \text{Node.val} \leq 100$
- d) $1 \leq n \leq sz$

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Amazon 3

Apple 3

Adobe 2

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Facebook **16** Google **4** Microsoft **4** Bloomberg **4** Yahoo **2**

eBay **2** Nvidia **2** VMware **2** American Express **2** Intel **2**

Dell **2**

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Uber **3** Walmart Global Tech **3** Oracle **2** Yandex **2** Salesforce **2**

Paypal **2** Nagarro **2**

19. Remove Nth Node From End of List - Solution

```

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

class Solution {
    public ListNode removeNthFromEnd(ListNode headNode, int n) {

        ListNode currNode = headNode;

        for(int i=1;i<=n && currNode!=null;i++) {
            currNode = currNode.next;
        }
    }
}

```



```
        if(currNode==null) {  
            return headNode.next;  
        }  
  
        ListNode prevNode = headNode;  
        while(currNode.next!=null) {  
            prevNode = prevNode.next;  
            currNode = currNode.next;  
        }  
  
        prevNode.next = prevNode.next.next;  
  
        return headNode;  
    }  
}
```